CS 419: Secure Web Application Project

Spring 2026

Detailed Project Specifications

Project Overview

Due Date: April 15th, 2026
Team Size: 2 students
Total Points: 200 points (20% of course grade)
Skills Covered: Authentication, Access Control, Input Validation, Encryption, Session Management, Security Headers, Logging

---

Secure Document Sharing System

Build a secure system for sharing confidential documents between users with access controls and encryption.

Core Features:

- User registration and authentication
- Upload/download encrypted documents
- Share documents with specific users
- Access control (owner, editor, viewer roles)
- Document versioning and audit trail

Technical Requirements

1. Technology Stack

Backend (Choose ONE):

- **Python/Flask** (Recommended for beginners)
- **Node.js/Express** (Good for real-time features)

Frontend:

- HTML5, CSS3, JavaScript
- Optional: Bootstrap or Tailwind CSS for styling

Data Persistence (NO DATABASE REQUIRED):

- **File-based storage** using JSON files
- **Structured file organization:**

```
data/
  users.json        # User accounts
  sessions.json      # Active sessions
  [feature].json     # Feature-specific data
  logs/
    security.log     # Security events
    access.log       # Access logs
```

Encryption Libraries:

- Python: `cryptography`, `bcrypt`, `PyJWT`
- Node.js: `crypto`, `bcrypt`, `jsonwebtoken`

---

2. Security Requirements (100 points)

A. User Authentication (15 points)

Registration :

```
# Requirements:
- Username: 3-20 characters, alphanumeric + underscore
- Email: Valid email format
- Password: Minimum 12 characters, complexity requirements:
  * At least 1 uppercase letter
```

```
  * At least 1 lowercase letter
  * At least 1 number
  * At least 1 special character (!@#$%^&*)
- Password confirmation must match
- Check for duplicate username/email
```

Login :

```
# Requirements:
- Hash passwords with bcrypt (cost factor >= 12) or Argon2
- Never store plaintext passwords
- Implement account lockout after 5 failed attempts (15 minutes)
- Rate limiting: Max 10 login attempts per IP per minute
- Session creation on successful login
- Log all authentication attempts (success/failure)
```

Example Implementation:

```python
import bcrypt
import json
import time

def register_user(username, email, password):
    # Validate inputs
    if not validate_username(username):
        return {"error": "Invalid username"}

    if not validate_password_strength(password):
        return {"error": "Password does not meet requirements"}

    # Hash password
    salt = bcrypt.gensalt(rounds=12)
    hashed = bcrypt.hashpw(password.encode('utf-8'), salt)

    # Store user (file-based)
    user = {
        "username": username,
        "email": email,
```

```
    "password_hash": hashed.decode('utf-8'),
    "created_at": time.time(),
    "role": "user",
    "failed_attempts": 0,
    "locked_until": None
}

save_user(user)
return {"success": True}
```

B. Access Control (15 points)

Role-Based Access Control (RBAC) :

Implement at least THREE roles:

1. **Admin** - Full system access
2. **User** - Standard access
3. **Guest** - Limited read-only access

Permission Matrix:

Feature	Admin	User	Guest
Create content	✓	✓	✗
Edit own content	✓	✓	✗
Delete own content	✓	✓	✗
View all content	✓	✗	✗
Manage users	✓	✗	✗
View shared content	✓	✓	✓

Authorization Checks :

python
```python
from functools import wraps

def require_auth(f):
    @wraps(f)
    def decorated_function(*args, **kwargs):
        if 'user_id' not in session:
            return redirect('/login')
        return f(*args, **kwargs)
    return decorated_function

def require_role(role):
    def decorator(f):
        @wraps(f)
        def decorated_function(*args, **kwargs):
            user = get_current_user()
            if user['role'] != role:
                abort(403)  # Forbidden
            return f(*args, **kwargs)
        return decorated_function
    return decorator

# Usage:
@app.route('/admin/dashboard')
@require_auth
@require_role('admin')
def admin_dashboard():
    return render_template('admin.html')
```

C. Input Validation & Injection Prevention (20 points)

Cross-Site Scripting (XSS) Prevention :

python
```python
import html
```

```python
def sanitize_input(user_input):
    """Escape HTML special characters"""
    return html.escape(user_input)

def sanitize_output(data):
    """Sanitize before rendering"""
    if isinstance(data, str):
        return html.escape(data)
    return data

# In templates (using Jinja2 auto-escaping):
# {{ user_input }}  # Automatically escaped
# {{ user_input | safe }}  # ONLY if you're CERTAIN it's safe
```

Command Injection Prevention :

python
```python
# NEVER do this:
import os
filename = request.form['filename']
os.system(f"cat {filename}")  # DANGEROUS!

# Instead, validate and use safe functions:
import re
import os.path

def safe_filename(filename):
    # Remove path traversal attempts
    filename = os.path.basename(filename)
    # Allow only alphanumeric, dash, underscore, dot
    if not re.match(r'^[\w\-\.]+$', filename):
        raise ValueError("Invalid filename")
    return filename
```

Path Traversal Prevention :

python
```python
import os
```

```python
from werkzeug.utils import secure_filename

def safe_file_path(user_path, base_dir):
    # Secure the filename
    filename = secure_filename(user_path)

    # Construct full path
    full_path = os.path.join(base_dir, filename)

    # Verify it's within base directory
    if not os.path.abspath(full_path).startswith(os.path.abspath(base_dir)):
        raise ValueError("Path traversal detected")

    return full_path
```

Input Validation Rules :

- Whitelist validation (allow known good, not block known bad)
- Length limits on all inputs
- Type checking (integers, emails, URLs)
- Sanitize file uploads (check extensions, MIME types, scan for malware)

D. Encryption (15 points)

Transport Encryption :

python
```python
# For development, generate self-signed certificate:
# openssl req -x509 -newkey rsa:4096 -nodes -out cert.pem -keyout key.pem
-days 365

# Flask with TLS:
if __name__ == '__main__':
    app.run(ssl_context=('cert.pem', 'key.pem'),
            host='0.0.0.0',
            port=5000)
```

```
# Force HTTPS:
@app.before_request
def require_https():
    if not request.is_secure and app.env != "development":
        url = request.url.replace("http://", "https://", 1)
        return redirect(url, code=301)
```

Data-at-Rest Encryption:

python

```
from cryptography.fernet import Fernet
import json

class EncryptedStorage:
    def __init__(self, key_file='secret.key'):
        # Load or generate encryption key
        try:
            with open(key_file, 'rb') as f:
                self.key = f.read()
        except FileNotFoundError:
            self.key = Fernet.generate_key()
            with open(key_file, 'wb') as f:
                f.write(self.key)

        self.cipher = Fernet(self.key)

    def save_encrypted(self, filename, data):
        """Save encrypted JSON data"""
        json_data = json.dumps(data)
        encrypted = self.cipher.encrypt(json_data.encode())

        with open(filename, 'wb') as f:
            f.write(encrypted)

    def load_encrypted(self, filename):
        """Load and decrypt JSON data"""
        with open(filename, 'rb') as f:
            encrypted = f.read()
```

```python
        decrypted = self.cipher.decrypt(encrypted)
        return json.loads(decrypted.decode())

# Usage:
storage = EncryptedStorage()

# Save sensitive data
storage.save_encrypted('data/passwords.enc', {
    'user1': {'site': 'example.com', 'password': 'encrypted_pass'}
})

# Load sensitive data
data = storage.load_encrypted('data/passwords.enc')
```

E. Session Management (15 points)

Secure Session Implementation :

python
```python
import secrets
import time
import json

class SessionManager:
    def __init__(self, timeout=1800):  # 30 minutes
        self.timeout = timeout
        self.sessions_file = 'data/sessions.json'

    def create_session(self, user_id):
        """Create new session token"""
        token = secrets.token_urlsafe(32)

        session = {
            'token': token,
            'user_id': user_id,
            'created_at': time.time(),
```

```
        'last_activity': time.time(),
        'ip_address': request.remote_addr,
        'user_agent': request.headers.get('User-Agent')
    }

    # Save session
    sessions = self.load_sessions()
    sessions[token] = session
    self.save_sessions(sessions)

    return token

def validate_session(self, token):
    """Check if session is valid"""
    sessions = self.load_sessions()

    if token not in sessions:
        return None

    session = sessions[token]

    # Check timeout
    if time.time() - session['last_activity'] > self.timeout:
        self.destroy_session(token)
        return None

    # Check IP address (optional - may break with mobile users)
    # if session['ip_address'] != request.remote_addr:
    #     return None

    # Update last activity
    session['last_activity'] = time.time()
    sessions[token] = session
    self.save_sessions(sessions)

    return session
```

```python
    def destroy_session(self, token):
        """Delete session"""
        sessions = self.load_sessions()
        if token in sessions:
            del sessions[token]
            self.save_sessions(sessions)

# Flask integration:
@app.before_request
def load_user_session():
    token = request.cookies.get('session_token')
    if token:
        session_data = session_manager.validate_session(token)
        if session_data:
            g.user_id = session_data['user_id']
        else:
            g.user_id = None
    else:
        g.user_id = None

@app.route('/login', methods=['POST'])
def login():
    # ... authenticate user ...

    # Create session
    token = session_manager.create_session(user_id)

    response = make_response(redirect('/dashboard'))
    response.set_cookie(
        'session_token',
        token,
        httponly=True,      # Prevent JavaScript access
        secure=True,        # HTTPS only
        samesite='Strict',  # CSRF protection
        max_age=1800        # 30 minutes
    )
    return response
```

F. Security Headers (10 points)

Required Headers :

python
```python
@app.after_request
def set_security_headers(response):
    # Content Security Policy
    response.headers['Content-Security-Policy'] = (
        "default-src 'self'; "
        "script-src 'self' 'unsafe-inline'; "  # Avoid unsafe-inline in production
        "style-src 'self' 'unsafe-inline'; "
        "img-src 'self' data:; "
        "font-src 'self'; "
        "connect-src 'self'; "
        "frame-ancestors 'none'"
    )

    # Prevent clickjacking
    response.headers['X-Frame-Options'] = 'DENY'

    # Prevent MIME type sniffing
    response.headers['X-Content-Type-Options'] = 'nosniff'

    # XSS Protection (legacy, but still useful)
    response.headers['X-XSS-Protection'] = '1; mode=block'

    # Referrer Policy
    response.headers['Referrer-Policy'] = 'strict-origin-when-cross-origin'

    # Permissions Policy
    response.headers['Permissions-Policy'] = (
        'geolocation=(), microphone=(), camera=()'
    )

    # HSTS (HTTP Strict Transport Security)
```

```
    response.headers['Strict-Transport-Security'] = (
        'max-age=31536000; includeSubDomains'
    )

    return response
```

Test Headers: Use https://securityheaders.com to verify implementation

---

G. Logging & Monitoring (10 points)

Security Event Logging :

python
```
import logging
import json
from datetime import datetime

class SecurityLogger:
    def __init__(self, log_file='logs/security.log'):
        self.logger = logging.getLogger('security')
        self.logger.setLevel(logging.INFO)

        handler = logging.FileHandler(log_file)
        formatter = logging.Formatter(
            '%(asctime)s - %(levelname)s - %(message)s'
        )
        handler.setFormatter(formatter)
        self.logger.addHandler(handler)

    def log_event(self, event_type, user_id, details, severity='INFO'):
        """Log security event"""
        log_entry = {
            'timestamp': datetime.utcnow().isoformat(),
            'event_type': event_type,
            'user_id': user_id,
            'ip_address': request.remote_addr,
            'user_agent': request.headers.get('User-Agent'),
```

```
            'details': details,
            'severity': severity
        }

        if severity == 'CRITICAL':
            self.logger.critical(json.dumps(log_entry))
        elif severity == 'ERROR':
            self.logger.error(json.dumps(log_entry))
        elif severity == 'WARNING':
            self.logger.warning(json.dumps(log_entry))
        else:
            self.logger.info(json.dumps(log_entry))

# Usage examples:
security_log = SecurityLogger()

# Login attempt
security_log.log_event(
    'LOGIN_SUCCESS',
    user_id=user_id,
    details={'username': username}
)

# Failed login
security_log.log_event(
    'LOGIN_FAILED',
    user_id=None,
    details={'username': username, 'reason': 'Invalid password'},
    severity='WARNING'
)

# Access denied
security_log.log_event(
    'ACCESS_DENIED',
    user_id=user_id,
    details={'resource': '/admin/users', 'reason': 'Insufficient privileges'},
    severity='WARNING'
```

```
)

# Data access
security_log.log_event(
    'DATA_ACCESS',
    user_id=user_id,
    details={'resource': 'document_123.pdf', 'action': 'download'}
)

# Account locked
security_log.log_event(
    'ACCOUNT_LOCKED',
    user_id=user_id,
    details={'reason': '5 failed login attempts'},
    severity='ERROR'
)
```

Events to Log:
- Authentication attempts (success/failure)
- Authorization failures
- Password changes
- Account lockouts
- Data access (create, read, update, delete)
- Session creation/destruction
- Security configuration changes
- Input validation failures
- Suspicious activity (rapid requests, invalid tokens)

Project Deliverables (200 Points Total)

1. Working Application (100 points) - See Technical requirements Above

Functionality :
- All core features working
- No critical bugs

- Meets all security requirements
- User-friendly interface

Code Quality:
- Clean, readable code
- Proper comments
- Organized file structure
- Follows Python/JavaScript best practices

Security Implementation :
- All security controls properly implemented
- No obvious vulnerabilities
- Secure by default

2. Security Design Document (40 points)

A. Architecture Overview (10 points)
1. System architecture diagram
2. Data flow diagrams
3. Component descriptions
4. Technology stack justification

B. Threat Model (10 points)
1. Asset identification
2. Threat enumeration
3. Vulnerability assessment
4. Attack scenarios
5. Risk prioritization

C. Security Controls (10 points)
For each security requirement, document:
1. Control description
2. Implementation approach
3. Testing methodology
4. Known limitations
5. Mitigation strategies

D. Data Protection (10 points)
1. Data classification
2. Encryption methods
3. Key management
4. Secure deletion procedures

10-15 pages

3. Penetration Testing Report (40 points)

Test Categories (Provide checklist):

A. Authentication Testing (10 points)
- [ ] Brute force protection
- [ ] Password complexity enforcement
- [ ] Session management
- [ ] Logout functionality
- [ ] Password reset security

B. Authorization Testing (8 points)
- [ ] Horizontal privilege escalation
- [ ] Vertical privilege escalation
- [ ] Direct object reference
- [ ] Forced browsing

C. Input Validation Testing (10 points)
- [ ] XSS attempts
- [ ] Path traversal
- [ ] Command injection
- [ ] File upload vulnerabilities

D. Session Security Testing (7 points)
- [ ] Session fixation
- [ ] Session hijacking
- [ ] Concurrent session handling
- [ ] Session timeout

E. Configuration Testing (5 points)
- [ ] Security headers
- [ ] TLS/SSL configuration
- [ ] Error handling
- [ ] Debug mode disabled

Report Format:

1. Executive Summary
2. Testing Methodology
3. Findings (Critical, High, Medium, Low)
   - Description
   - Impact
   - Steps to reproduce
   - Proof of concept
   - Remediation recommendations
4. Conclusion

4. Presentation (20 points)

15-minute presentation (ppt or slide) 5-8 slides covering:

1. Project overview
2. Security architecture
3. Key security implementations
4. Challenges and lessons learned

PROPOSED Project Timeline & Milestones:

Week 1: Planning & Setup
- Define team and roles
- Set up development environment
- Create project repository
- Ask clarifying questions

Week 2: Core Functionality
Basic working prototype

- User registration/login
- Basic feature implementation (no security yet)
- File structure setup

Week 3-4: Security Implementation

Security controls implemented
- Authentication & access control
- Input validation
- Encryption
- Session management

Week 5: Security Headers & Logging

Complete security implementation
- All security headers configured
- Logging system operational
- Security design document draft

Week 6: Testing & Documentation

Penetration testing
- Self-testing
- Exchange with another team
- Document findings

Week 7: Refinement

Fix identified issues
- Address critical/high findings
- Code cleanup
- Final documentation

Week 8: Prepare final Presentation

- Prepare slides
- Submit the code

Submission Requirements

GitHub Repository Structure:

```
secure-app/
├── README.md              # Project overview, setup instructions
├── requirements.txt       # Python dependencies
├── app.py                 # Main application
├── config.py              # Configuration
├── data/                  # Data storage
│   ├── users.json
│   └── sessions.json
├── logs/                  # Log files
│   └── security.log
├── static/                # Static files
│   ├── css/
│   └── js/
├── templates/             # HTML templates
├── docs/                  # Documentation
│   ├── security_design.pdf
│   └── pentest_report.pdf
├── tests/                 # Test scripts
└── presentation/          # Presentation slides
```

Submission Checklist:

- GitHub repository with all code
- Security design document (PDF)
- Penetration testing report (PDF)
- Presentation slides (PDF/PPTX)
- README with setup instructions
- Demo video (5 minutes, optional)

Resources & Tools

Development:

- Flask documentation: https://flask.palletsprojects.com/
- Express.js: https://expressjs.com/
- Cryptography library: https://cryptography.io/

Security Testing:

- OWASP ZAP: https://www.zaproxy.org/
- Burp Suite (Community): https://portswigger.net/burp
- SecurityHeaders.com: https://securityheaders.com/

References:

- OWASP Top 10: https://owasp.org/www-project-top-ten/
- OWASP Testing Guide: https://owasp.org/www-project-web-security-testing-guide/

Common Pitfalls to Avoid

❌ **Storing passwords in plaintext**
✅ Use bcrypt with cost factor >= 12

❌ **Weak session tokens**
✅ Use `secrets.token_urlsafe(32)`

❌ **Not validating inputs**
✅ Validate, sanitize, and escape all user inputs

❌ **Missing security headers**
✅ Implement all required headers

❌ **Poor error handling exposing system info**
✅ Generic error messages to users, detailed logs for admins

❌ **Hardcoded secrets in code**
✅ Use environment variables or config files (not in git)

❌ **No logging**
✅ Log all security events

Academic Integrity

- All code must be original or properly attributed
- You may use example code, libraries and frameworks
- You may reference online tutorials, but must understand and explain your code
- Copying another team's code = automatic F on project
- AI assistance (ChatGPT, Copilot) allowed for learning, but you must understand and be able to explain every line